

How Hardware Gets Hacked (Part 9)

A practical security exercise

by Nathan Jones

Introduction	1
“Enable feature”: A practical security exercise.....	2
Design review of the “feature” feature.....	2
Security analysis of “features”	5
A possible attack and its defense (Attack/Defense #6).....	6
The “blast radius” problem again.....	11
In the driver’s seat.....	13
Updated threat model.....	15
Conclusion.....	16
Attacking and defending “start” messages.....	17

Introduction

In the [last article](#) we shifted our attention away from the unlocking process (which would seem to have required some rather sophisticated attacks after Part 7) to the pairing process, which was vulnerable to some very simple brute force and timing attacks. The last thing left to secure for the competition is the process of **packaging and enabling a feature**.

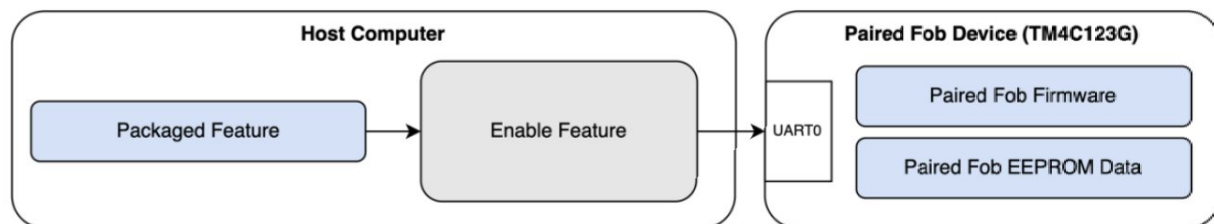


Figure 7. Enable Feature Step

As with pairing, this code is woefully insecure at the moment. But the good news is that we already have all the skills we need to make it secure! In this article, I’ll ask you to apply

what you’ve learned so far in this series to do exactly that, walking you through my own solution. Before we close, we’ll also get to talk about per-device keys and asymmetric cryptography.

“Enable feature”: A practical security exercise

Let’s put those security skills you’ve been developing in this series to the test! Can you protect the “enable feature” aspect of our system from exploitation?

We’ll tackle this challenge in three steps:

1. **Reviewing** how features are packaged, formatted, and enabled
2. Identifying the **security requirements** our system needs
3. Demonstrating a **successful attack** on the system (by exploiting a security requirement which is lacking enforcement) and then implementing a **defense** against that attack

Each step is composed of several questions which I hope you take the time to answer (even if it’s just briefly and in your own head!) before I provide you with my answer.

Design review of the “feature” feature

/*****[? How are features packaged?]***/

Before a feature can be enabled, it must be “packaged” into a feature file. What is the shell command that packages a feature for our system? Who is responsible for executing that command? How were these features distributed to the teams in the competition?

Hint: You can find this information in either the [repo README](#), [HHGH \(Part 2\): On-boarding](#), and/or the [competition rules](#).

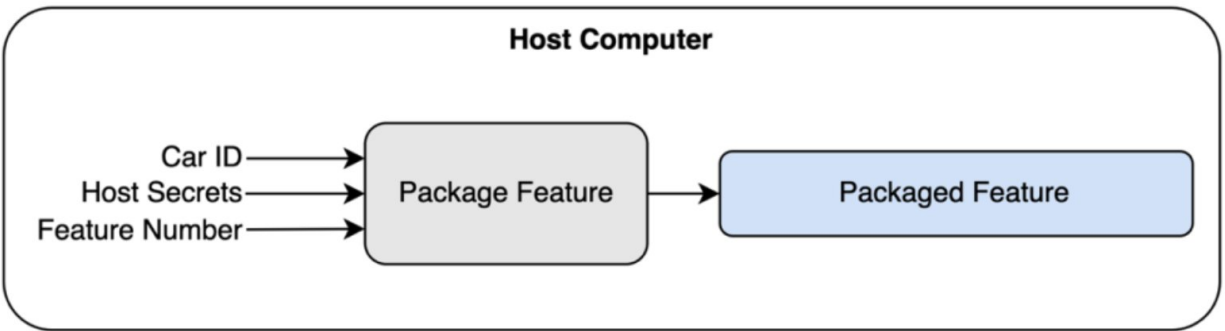
*****/

↓ ANSWER BELOW ↓

Features are packaged by executing the following command in the terminal.

```
# Packages Feature 1 for Car 1234
./tools/package.py --id 1234 --num 1
```

In the competition, the `package.py` tool also had read/write access to any “host secrets” (e.g. `secrets.json`).



Features are packaged at the “factory” (i.e. by the MITRE competition organizers) and were distributed to attacking teams alongside the encrypted firmware binaries for each of the car scenarios (except for Car #5; more on this in just a bit).

✓ = Full Access ✓ = Temporary Access	Car	Paired Fob	Unpaired Fob	Logic Analyzer Capture of Unlock	Pairing PIN	Packaged Feature 1	Packaged Feature 2
Car 0 Your Car (No Flags)	✓	✓	✓		✓	✓	✓
Car 1 New Car	✓		✓			✓	✓
Car 2 Temporary Fob Access	✓	✓	✓			✓	✓
Car 3 Passive Unlock	✓		✓	✓		✓	✓
Car 4 Leaked Pairing PIN	✓		✓		✓	✓	✓
Car 5 PIN Extraction, Enable Feature	✓	✓	✓			✓	

/*****[? What is the format of a feature file?]***/

What did a feature file actually look like? What is actually being saved to disk or sent over the UART port when a feature is being packaged or enabled?

Hint: You can find this information in either the [repo README](#), [HHGH \(Part 1\)](#), or by [reviewing the code](#).

*****/

↓ ANSWER BELOW ↓

Feature files are simply binary files composed of the car ID and a feature number.

FEATURE PACKET:

Car ID (11 bytes)	Feature # (1 byte)
-------------------	--------------------

/*****[? How are features enabled on a fob?]***/

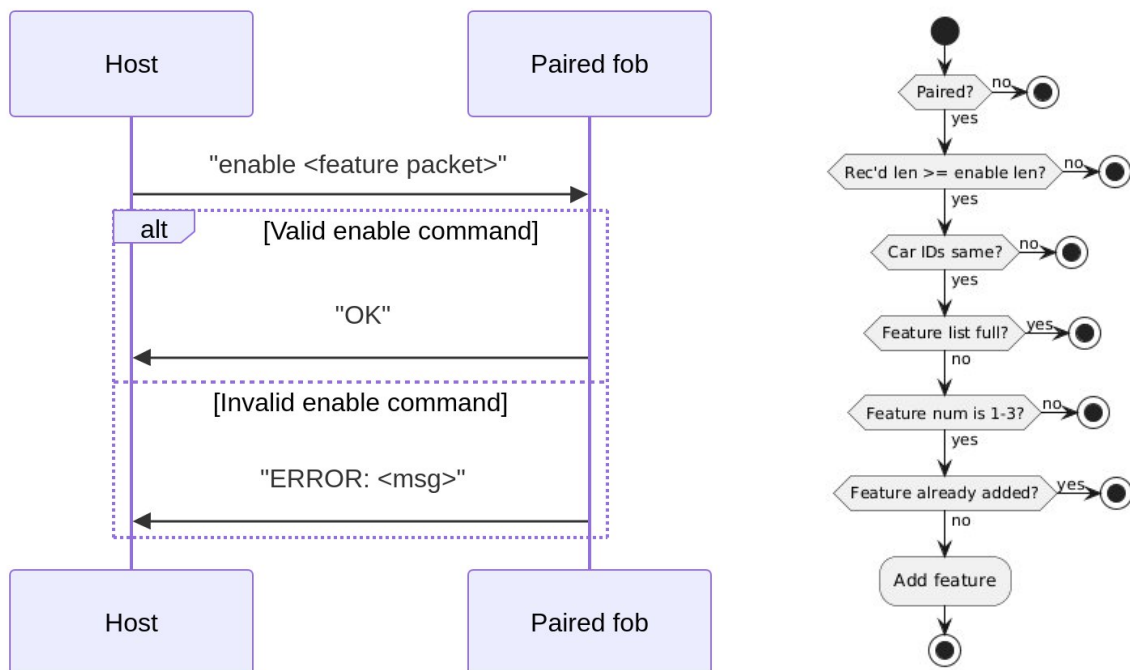
What initiated the process of enabling a feature? What steps were taken by the fob before that feature was possibly enabled?

Hint: You can find this information in either the [repo README](#) or [HHGH \(Part 1\)](#).

*****/

↓ ANSWER BELOW ↓

Features were enabled on a paired fob by sending it the ASCII message enable <feature>\n to the HOST UART port, where <feature> is the Base16-encoded contents of the binary feature file. (Base16 encoding converts binary or hex values like 0b10011111 (0x9F) (0x39 0x46), which makes it easier to read and debug later on when looking at a transcript of what was sent to a fob.)



An unpaired fob would reject the message straightaway. A paired fob would check that the feature file was valid (length was greater than or equal to the expected value, car ID in the feature file matched it's own car ID, feature number within range) and that it could be

enabled (feature list wasn't full, feature wasn't already enabled) before enabling it, sending back OK or ERROR (with an error message) as appropriate.

Security analysis of “features”

Now that we have a thorough understanding of the pairing process, we can begin to analyze what security requirements must be present to meet the competition rules/our design goals.

```
/*****[? What flags are at stake?]*****/
```

What flags could an attacking team capture by abusing the “enable feature” part of our system? What things should our system prevent a malicious person from doing?

```
*****/
```

↓ ANSWER BELOW ↓

In the MITRE eCTF competition, teams could capture a flag if they could cause Car #5 to emit its flag for “Feature 2” (which wasn't given to the attacking teams; see the graphic above of the attack scenarios) and which was only *supposed* to be possible if the paired fob for Car #5 had Feature 2 legitimately enabled.

More broadly, features should only ever be made for owners who've paid for them, car owners should not be able to enable a feature without a valid feature file from the manufacturer, and they should also not be able to enable a feature on their car using a valid feature file that was created by the manufacturer for a different car (these are, essentially, security requirements #5 and #6 from the competition rules).

In a real production system, you may also want to ensure that feature files are unique to certain drivers or that feature files eventually “expire” or are “one-time use”.

```
/*****[? What security requirements do we need?]*****/
```

In [Part 5](#) we listed 5 important security principles: confidentiality, integrity, availability, authentication, and non-repudiation. Which, specifically, do we need and might possibly be missing from our current implementation? (Hint: there may be more than one!)

```
*****/
```

↓ ANSWER BELOW ↓

If a fob needs to enforce that a feature file which it has received has come from none other than the manufacturer themselves, then they are needing to **authenticate** that feature file. Additionally, if it's important that a valid feature file not be modified by an attacker to

enable a feature on a different car or to enable a different feature on the same car, then the fob needs a way to verify the **integrity** of each feature file.

A possible attack and its defense (Attack/Defense #6)

Knowing what security requirements we need, let's discuss how an attacker can exploit the fact that we're not doing anything right now to enforce those security requirements and then show how to enforce those requirements and close off the attacks.

/*****[? What is an attack that could exploit our current system?]*****/

What is the easiest possible attack you can think of to capture the "Feature 2" flag from Car #5 by exploiting our current system?

*****/

↓ ANSWER BELOW ↓

The easiest attack that could be conducted against a feature file is **forgery**. Forging a feature file is quite simple, since the feature files are simply binary files composed of the car ID and a feature number.

FEATURE PACKET:

Car ID (11 bytes)	Feature # (1 byte)
-------------------	--------------------

We can easily modify either field to be whatever we want, allowing us to enable *any* feature on *any* car.

We'll call this "**Attack #6: Forging a feature file**".

/*****[? What would a security test for this attack look like?]*****/

How would you write the test to go in `test_security.py` that implements this attack? Write it out in Python (or in pseudocode) and then run it to make sure that it fails, as expected!

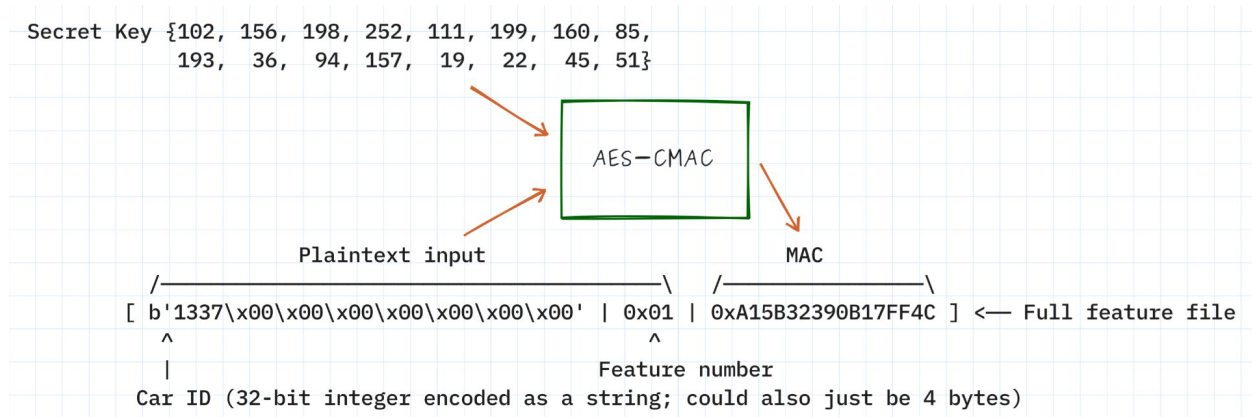
*****/

↓ ANSWER BELOW ↓

The [failing test that I wrote](#) forges a feature file by performing the following actions.

1. Pull the data off a fob (to ensure feature 2 isn't enabled yet),
2. Package feature 1,

Actually, we've already seen the best defense for this attack: **computing a MAC value** over the feature file and appending that to each feature packet.



The most important property that a MAC algorithm actually has is “resistance to forgery”, which provides both authenticity and integrity: an attacker could neither create their own valid feature message nor could they modify a valid feature message and have it remain valid. In either case, they wouldn’t be able to compute the correct MAC value without knowing the secret key.

Previously we used MAC values to authenticate the unlock messages from a paired fob. In that case, the MAC value gave us integrity in addition to authenticity, though integrity wasn’t necessarily a requirement of the design. The only thing an attacker could do by modifying an unlock message was have it rejected by the car.

Our new feature file will have a MAC value appended to the end, computed over the car ID and feature number using a new “feature key” (which every fob will get at build-time). The new format looks like this:

FEATURE PKT:

Car ID (11 bytes)	Feature # (1 byte)	MAC value (8 bytes)
-------------------	--------------------	---------------------

/*****[? Why not use the unlock key?]***/

You might be asking, at this point, “There’s already a secret key on the fob; why can’t we use that?”

The short answer is that we *could*, but it’s considered bad practice. Much like toothbrushes, everybody should get their own; meaning, it’s best to only use each secret key for a single purpose. Although outside the scope of this series, it’s possible for an attacker to gain information about a key if it’s reused for multiple purposes. And if they were able to compute the key value, then they would have access to both forging a new feature file *and* impersonating a fob during the unlock process. If, instead, we use separate keys then the

damage isn't as bad if they happen to correctly compute one of them. This is known as **key separation**.

An alternative to storing unique keys on the fob for each unique purpose is to store one key and *derive* any other keys from it (**key derivation**). For instance, a fob could get a single `fob_key` and then compute the unlock and feature keys on-the-fly using the HKDF algorithm. HKDF is composed of two operations: Extract and Expand, the first of which can be skipped if the seed material is uniform (as would be the case if it were the output from a CSPRNG).

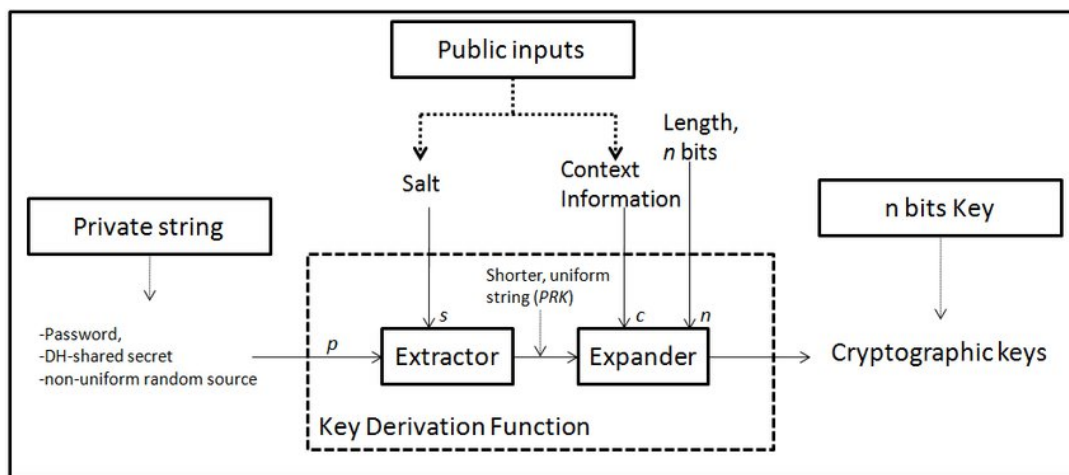
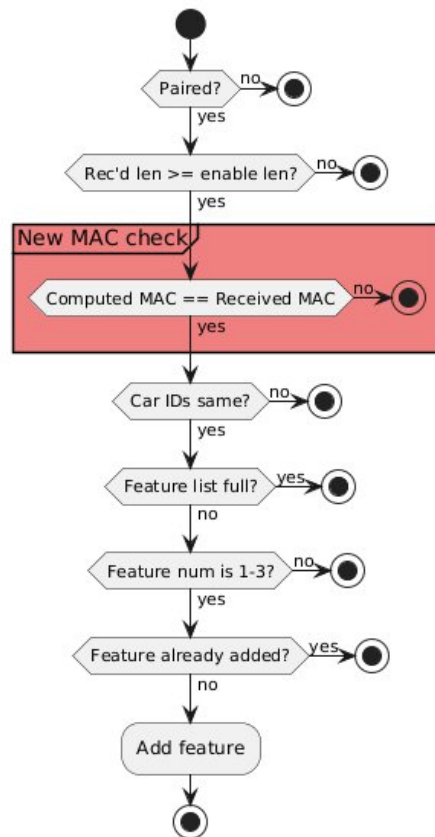


Figure 1: https://www.researchgate.net/figure/Extract-then-expand-model-for-KDFs_fig2_287478235

If keys are being derived from a `fob_key`, then `fob_key` should *never* be used in any other context (to ensure proper key separation).

*****/

The fob will test that this MAC value matches the one it computes over the first part of the message using its feature key before going on to check any of the other fields, failing early if it doesn't match.



Since an attacker doesn't have access to the feature key, they can't recompute a new MAC for a modified message nor can they construct new messages. Even a one bit change to either the car ID or the feature number causes the MACs to mismatch, which the fob will then reject.

We'll call this **"Defense #6: Authenticating feature files with MACs"**.

After implementing this defense, our test now passes!

```

> pytest testing/security_tests.py::TestFeatureFile::test_feature_file_cannot_be_modified_and_deployed
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 1 item

testing/security_tests.py .

===== 1 passed in 0.92s =====
  
```

There's a subtlety to this defense, though; a vulnerability we've seen before and need to make sure we keep closed in our new defense.

/*****[? What key property does the MAC comparison need to have?]***/

In other words, what attack is left open by the naive code below?

```
uint8_t computed_mac[16] = {0};
AES_CMAC_digest(&feature_cmac_ctx,
                (uint8_t*)enable_message,
                offsetof(ENABLE_PACKET, mac),
                computed_mac);
if( memcmp(&computed_mac[8], enable_message->mac, 8) != 0 )
{
    sendError("bad MAC");
    return;
}
```

Hint: The pairing pin suffered this attack until we fixed it!

*****/

↓ ANSWER BELOW ↓

When comparing MACs, don't forget to use a **constant-time memcmp** (i.e. `memcmp_ct`), like we did in [Part 8](#) when comparing pairing pins. Otherwise you'll leave yourself open to a timing attack!

/*****[! Attack it!]***/

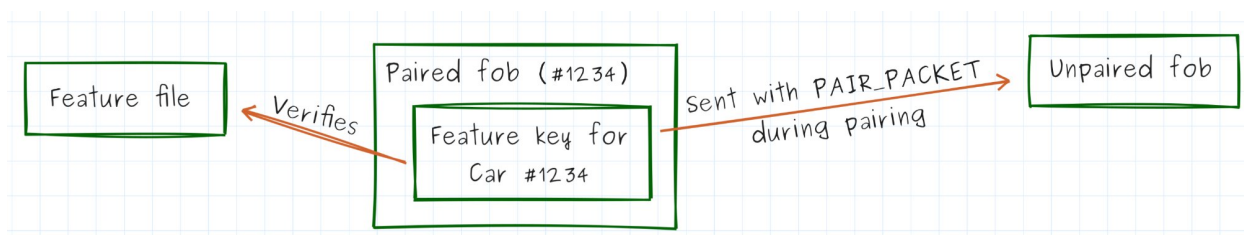
Write a test to conduct a timing attack on the naive code above and run it to prove that it's possible. Then switch out `memcmp` for `memcmp_ct` and prove that the code is no longer susceptible to a timing attack on the feature file's MAC value. ([One possible answer](#))

*****/

The “blast radius” problem again

Although our code is secure at this point, there are two modifications we could make that would help limit the negative effects of an attacker learning the value of “feature key” in the future. This is the “blast radius” problem that was mentioned in [Part 8](#). If an attacker is able to extract the “feature key” that's stored on *every fob* then they can have full control over the entire feature file verification system: they can forge whatever feature file they want for as long as that key remains in use. This would be a catastrophic failure of security, so instead we might use one of the techniques below (in addition to locking the debug port and doing our best to ensure that an attacker can't actually extract that “feature key”).

First, we could use a **per-device feature key**, as opposed to a company-wide one like we're using now. In this system, each paired fob would get a feature key that's unique to a single car ID, similar to how each paired fob gets an unlock key that's unique to each car ID. If an attacker ever extracts that feature key, the most damage they can do is forge feature files *for that specific car*, but not for every car we've ever produced.



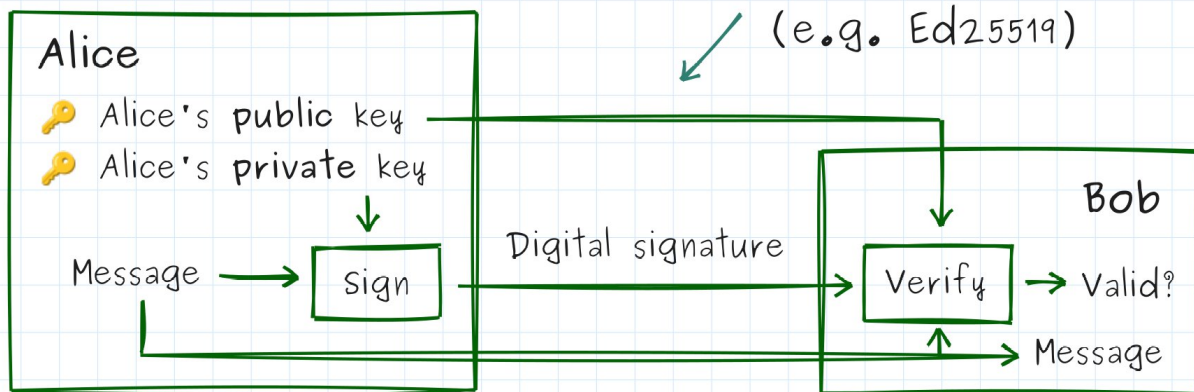
The downside is that our design gets a little more complex. Our company database (i.e. `secrets.json` in our codebase) would need to keep track of the feature key that's been associated with each car ID. Additionally, this key would need to be transmitted to an unpaired fob that was being paired as part of the `PAIR_PACKET` it gets sent.

If we wanted to reduce the blast radius even further, we could replace our MAC algorithm with a **digital signature**, which uses asymmetric cryptography, a.k.a. public-key cryptography. Asymmetric cryptography uses two keys, a “public” and a “private” key, as opposed to symmetric cryptography, which uses just one key. (We've been using symmetric cryptographic algorithms this whole time, as evidenced by the fact that our unlock and feature keys needed to be shared by all devices computing the same MAC values: one key on all devices.)

In asymmetric cryptography the private key is kept secret from *everybody* except the person or device holding the key while the public key can be distributed to anyone. The weird and neat thing about asymmetric cryptography is that you can't “double dip”: if the public key is used to sign/encrypt a message then only the private key can be used to decrypt it; trying to decrypt the message with the public key again just yields garbled-gook. Similarly, if a message is signed/encrypted with the private key then only the public key can decrypt it.

Asymmetric cryptography

↳ Encryption, Key exchange, Digital signature



That second scenario is essentially how a digital signature works: an entity performs a signing operation on a message with its private key, which anyone else can then verify using that same entity's public key. If the verify operation returns true, then it *had* to have been that actual entity that did the signing in the first place (since no one else could produce a valid signature to pass the verify operation without knowing the value of the private key). What's more, there's no shared secret that could get leaked if an attacker extracts a key from a device using the public key.

Using this system, we would generate a public/private key pair during build time and put the public key inside every fob. The private key would be used to sign any feature files, which the fobs could then verify. This system has the smallest blast radius (it's essentially none), since there's no key on the fob that could be extracted to allow an attacker the ability to forge a feature file.

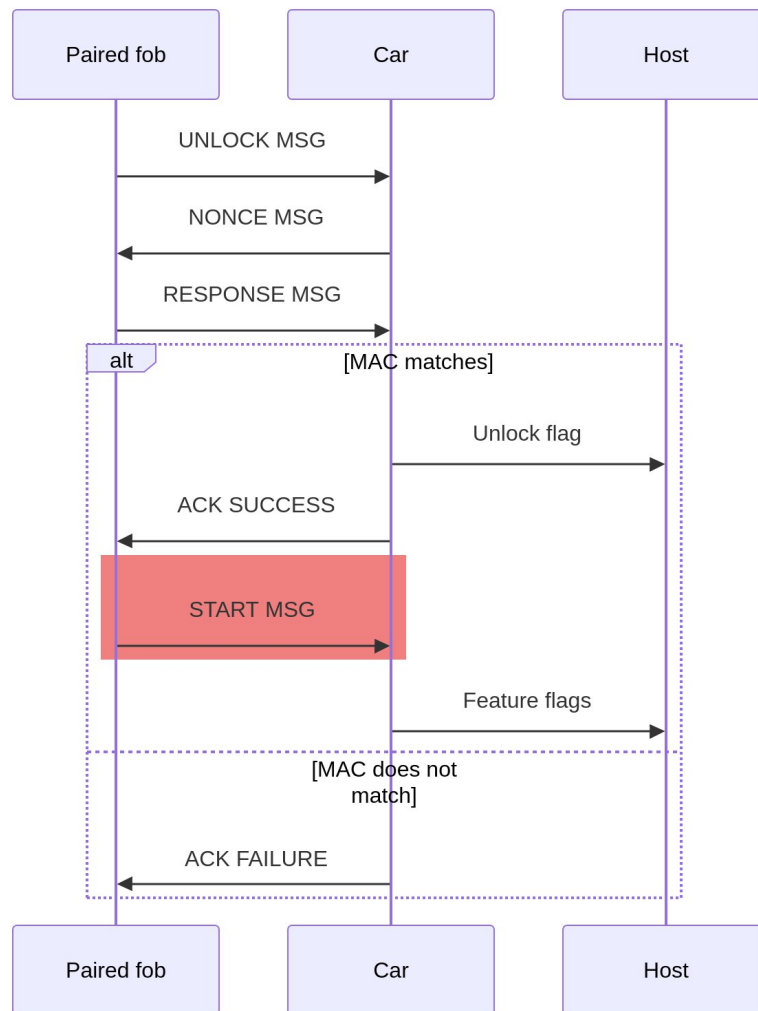
In the driver's seat

Now I'm going to step back even further, by letting you run through that same process to identify *where else* the feature data is vulnerable, to describe and mount the attack, and then to design and implement an appropriate defense.

To clarify, yes, there is one other place in the current code where an attacker could force a car to divulge its feature message despite not having that feature enabled on the fob. **But where?** (Take a second to consider your answer before moving on!)

↓ ANSWER BELOW ↓

Sending a feature file to a fob is only the first of two places where the feature data can be attacked. The second is when it's being transmitted as part of a start message after a car has been successfully unlocked!



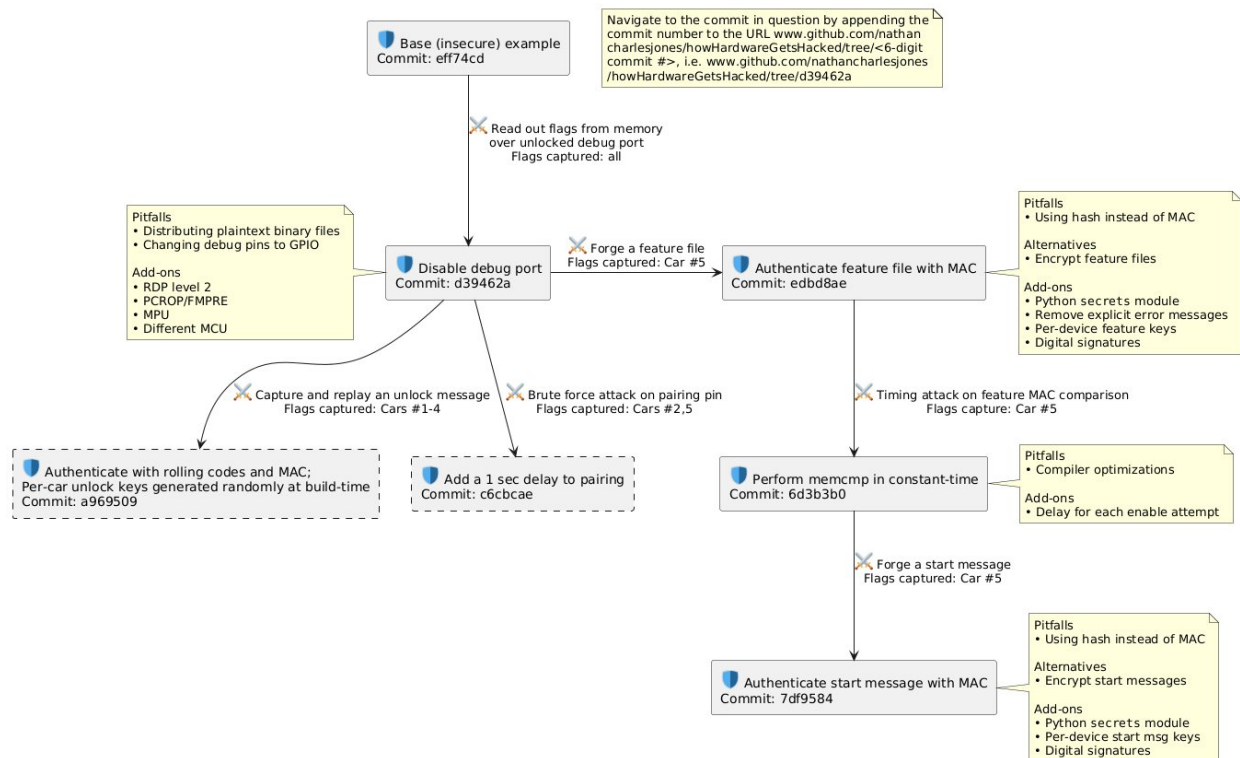
Each start message contains the fob's list of enabled features, and these are sent to the car in plaintext, with no additional security features whatsoever.

The process of attacking and securing this exchange is the same as what we've just walked through, even if the exact attack and defense steps are slightly different. I'll leave it to you to complete that process for this part of the design, all the way from "What's a start message and how is it used?" to "What's the easiest way to attack this and force Car #5 to spit out the Feature #2 flag?" and "How to protect against that attack?" If you get stuck, you can follow along with my explanation at the [end of this article](#). Good luck!

Updated threat model

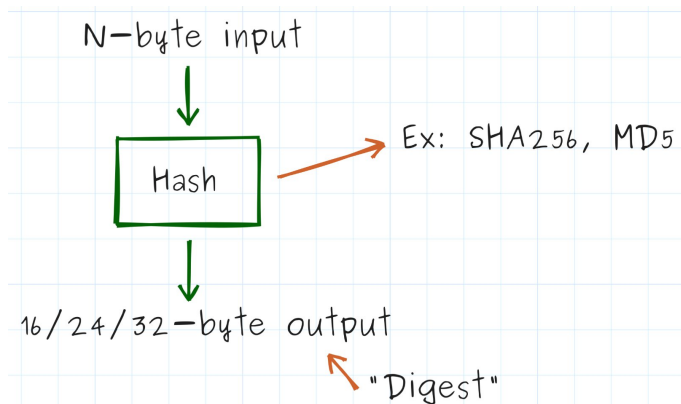
SPOILER ALERT: This section and the conclusion assume you've worked through the process of securing the start messages, the vulnerability identified in the last section.

The updated threat model (scoped just to show the latest attacks) is depicted below.



A few things worth noting that weren't previously discussed:

- One **pitfall** to watch out for is using an algorithm that provides integrity but not authentication. An example would be a hash, like SHA256 or MD5.



These algorithms could help a fob or car detect *accidental* errors in a feature file or

start message, but they wouldn't stop an attacker from changing a value and then computing a new hash (these are public algorithms, after all!).

- An alternative to adding a MAC value to each feature file or start message is to **encrypt them** using an authenticated encryption algorithm like AES-GCM. This would be only moderately more complicated than computing a MAC and would add “confidentiality” to our feature enabling process, but that’s not necessarily something we care about.
- The Python **secrets module** could be used to generate the keys instead of `randbytes` (previously mentioned when discussing how unlock keys were generated in [Part 6](#)).
- We could **remove the explicit chain of error messages** in `enableFeature`. If an attacker is able to somehow generate a valid MAC, then the rest of this function is set up to inform them exactly which part of their feature file (if any) is invalid for this specific fob. This is another example of a “side-channel leakage” (introduced in Part 8) and although it may not be exploitable today, it’s a thing to keep your eye on.
- **Compiler optimizations** could still remove the “constant-time” feature of our constant-time `memcmp`, `memcmp_ct`.
- An **added delay** during each enabling attempt would further help to defend against timing attacks or any other attack that relied on repeatedly and quickly attempting to enable a new feature.

Conclusion

In this article, we applied the security principles we’ve been developing in this series to the process of enabling new features on a fob.

Upon analyzing the firmware, it became clear that attackers could easily forge their own feature files *and* start messages, representing a lack of integrity and authentication in our current system. As the primary feature of a MAC (message authentication code) is “resistance to forgery”, appending one both to each feature file and to each start message cleanly solved the problem of forged messages. But don’t forget to make sure that you compare the MAC values using a constant-time `memcmp`! Otherwise you’ll leave yourself open to a timing attack. At this point, an attacker must find a way to defeat our MAC algorithm (an exceedingly hard thing to do) to be able to forge a feature file or a start message.

If you’ve made it this far, thanks for reading and happy hacking!

Attacking and defending “start” messages

We’re going to start this process of security analysis over again, walking gradually onto yet another important defense. As before, I want to emphasize that we’ve already discussed all of the tools you’d need to do this on your own and I want to encourage you to answer each question to yourself (even just in your head) before reading my answers.

Let’s start at the top.

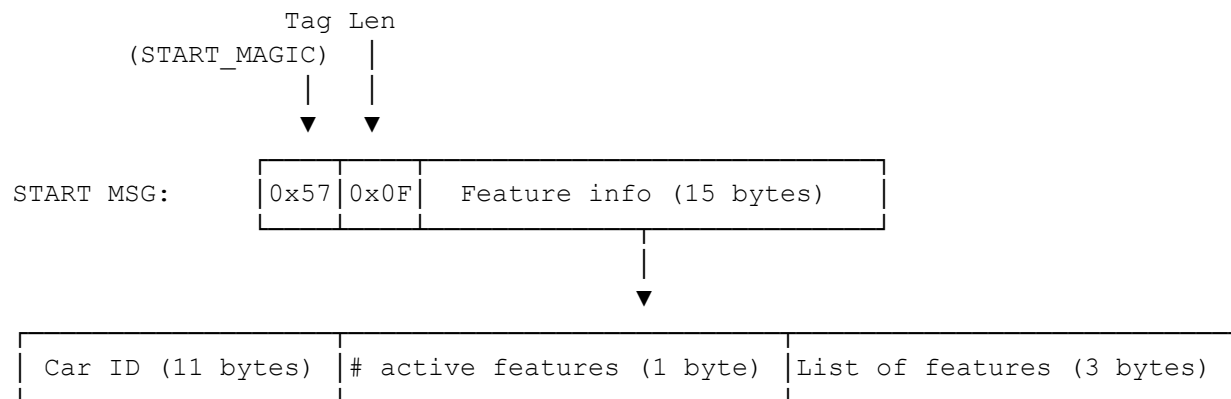
/*****[? What is the format of a start message?]***/

What does a start message actually look like? What is actually being sent over the UART port when a fob is telling a car which features have been enabled?

Hint: You can find this information in either the [repo README](#), [HHGH \(Part 1\)](#), or by [reviewing the code](#).

*****/

A start message consists of 17 bytes in TLV (tag-length-value) format:



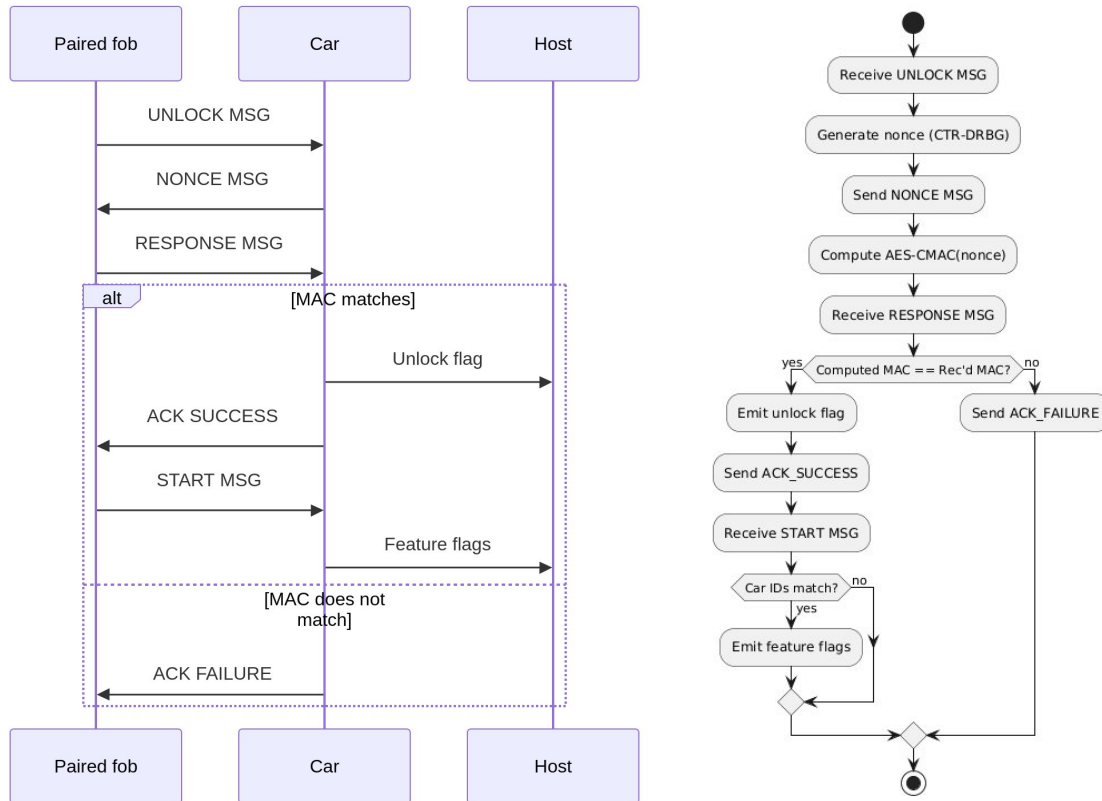
/*****[? How are active features sent from a fob to a car?]***/

When are features sent to the car? What steps are taken by the car before it prints out the associated feature flags?

Hint: You can find this information in either the [repo README](#) or [HHGH \(Part 1\)](#).

*****/

The fob sends active features to the car after it receives the ACK SUCCESS message from the car during the unlocking process.



The car checks only that the ID in the start message matches its own and that each active feature number is within a valid range (1-3 for us) before sending out the associated flag over its HOST UART port.

The same flag is at stake as before: the Feature 2 flag from Car #5 (which, again, doesn't have a valid Feature 2 file for it to be enabled legitimately).

/*****[? What security requirements do we need?]***/

Which of the five (confidentiality, integrity, availability, authentication, and non-repudiation) do we need and might possibly be missing from our current implementation? (Hint: there may be more than one!)

*****/

A car needs to know that the feature info it's getting from a fob has come from a valid paired fob and was untampered-with before it got to the car. This means we need authentication and integrity.

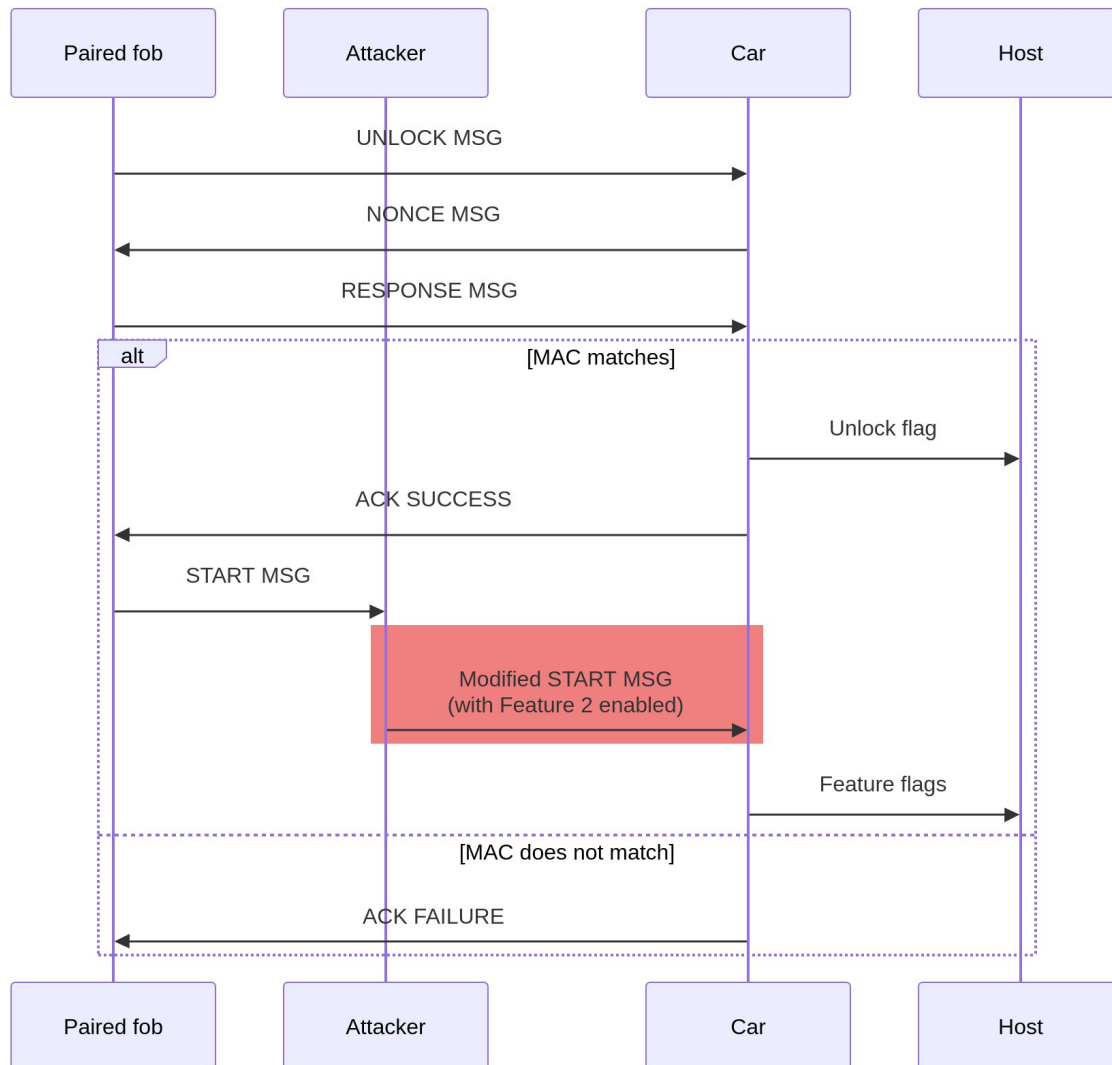
/*****[? What is an attack that could exploit our current system?]***/

What, now, is the easiest possible attack you can think of to capture the "Feature 2" flag from Car #5 by exploiting our current system?

Hint: It specifically targets the start message.

*****/

The easiest attack that could be conducted against a start message is still **forgery**, though forging a start message is a little more complicated than forging a feature file. The challenge is in the timing: we need Car #5's paired fob to begin the unlock process, getting as far as issuing a correct response to the car's nonce challenge, and then *we* need to send our own start message, instead of the one the fob wants to send.



This is a type of attack called **“man-in-the-middle”** (MITM), since the attacker literally sits in between the two devices trying to communicate, with the ability to modify the messages that are being passed back and forth.

Modifying (i.e. forging) a start message in this scenario is as easy as it was to forge a feature file (before we added the MAC, that is): all we have to do is change two binary

values in the start message (the number of active features and the feature or features which is/are active). This means that this attack would allow us to, once again, enable *any* feature on *any* car.

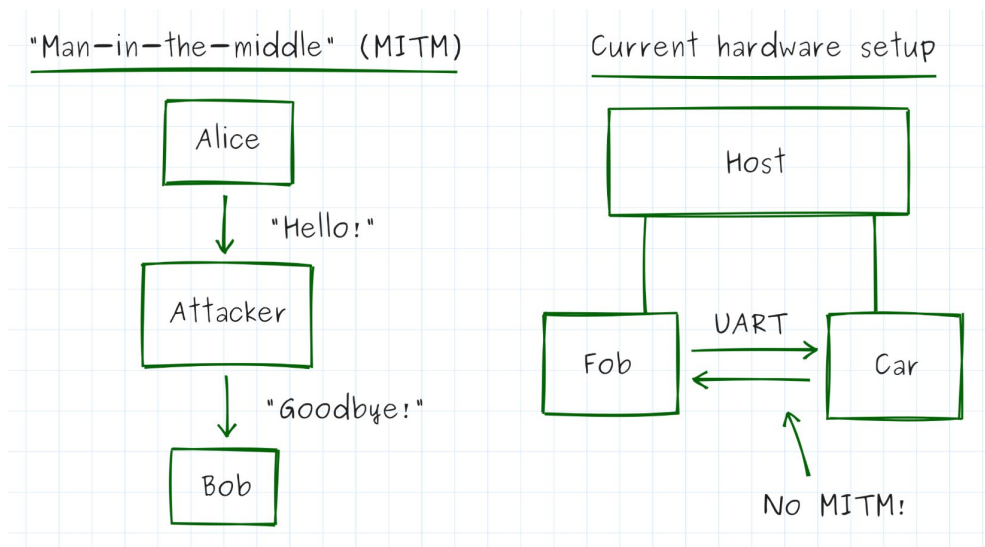
We'll call this **"Attack #7: Forging a start message"**.

```
/*****[? What would a security test for this attack look like?]***/
```

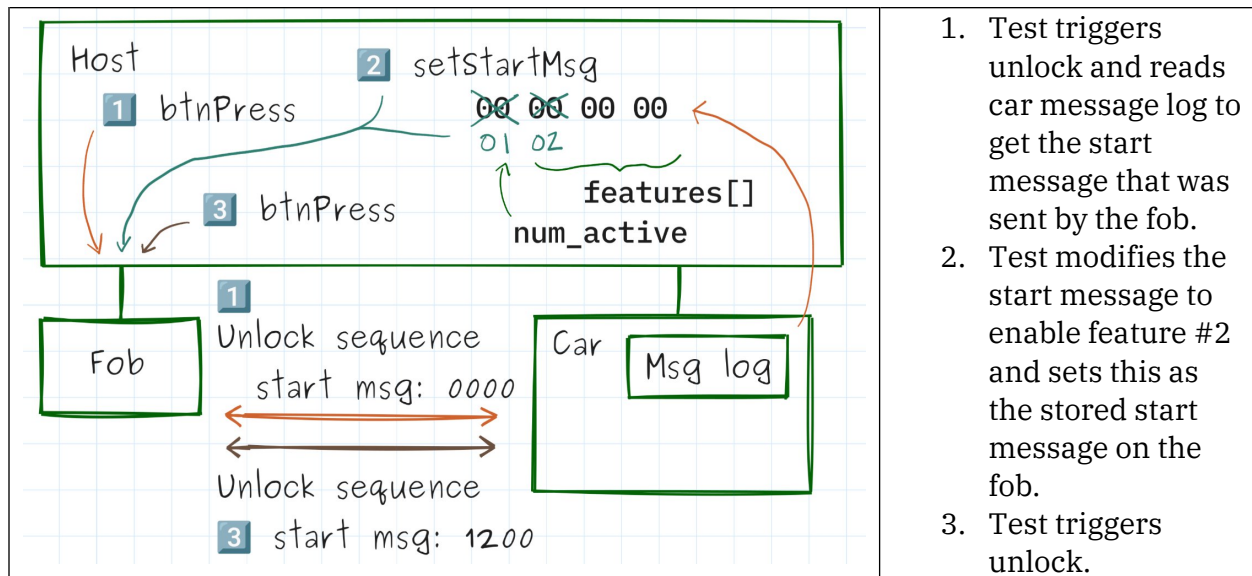
How would you write the test to go in `test_security.py` that implements this attack? Write it out in Python (or in pseudocode) and then run it to make sure that it fails, as expected!

```
*****/
```

The biggest challenge facing us in implementing this attack is that our current hardware setup has no way to "insert" an attacker in between a fob and a car to modify any start messages; they're directly connected to each other over UART. We don't have a way to conduct a real MITM-type attack.



So it's simulation to the rescue! To avoid needing to include a real "MITM" in our hardware setup, I've opted to create a new test command called [setStartMsg](#), which will store a binary start message in the fob to be sent out *in place of* its normal start message the next time the fob tries to unlock a car.



The [failing test that I wrote](#) first triggers an unlock and then extracts the last start message from the board message log (simulating a MITM attacker who can see a valid start message being transmitted by a fob). It then modifies the start message so that feature #2 is enabled and uses `setStartMsg` to save that modified message to the fob. Then it triggers an unlock, during which the fob uses the modified message.

To verify that the test currently fails, I've also added a car test command called [getFeatures](#) which lists the features that were sent over by a paired fob during the last valid unlock attempt.

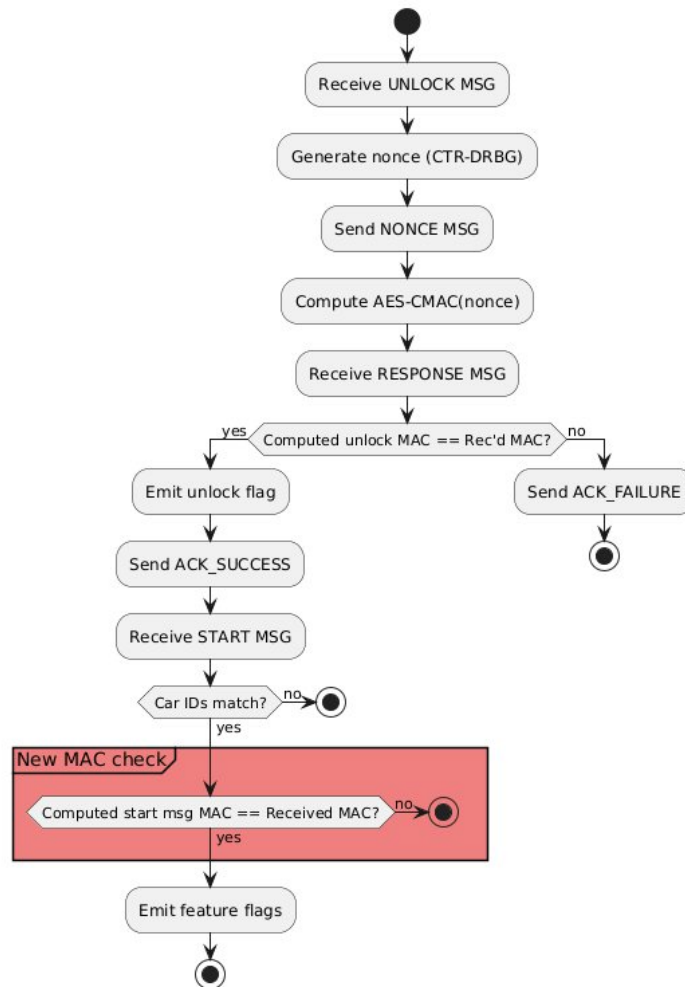
```
/*****[? How can we defend against that attack?]*****/
```

What's a way we can prevent an attacker from enabling a forged feature file?

Hint: We've already talked about how a car can authenticate an unlock message from a fob ([Parts 5-7](#)); how could you adapt or modify that solution to the problem of authenticating a start message?

```
*****/
```

Given the similarity of each of these attacks on the feature data, it will hopefully come as no surprise to learn that the easiest defense against this attack is to add a MAC to the end of each start message, which the car will also verify before it sends out any flags. If an attacker modifies even a single bit of the start message sent by the paired fob (or tries to forge their own), they won't be able to correctly compute the MAC value without the "start message" key.



We'll call this **"Defense #7: Authenticating start messages with MACs"**.

Implementing this requires [creating a new "start message" key](#) to go into each fob/car (which will also need to be added to the pair packet when pairing a new fob) and adding [one more conditional check](#) during the car's unlock process. Don't forget to use `memcmp_ct`!